

國立東華大學資訊工程學系

National Dong Hwa University

114 學年度大學部畢業專題研究報告

114 CSIE Undergraduate Project Report

HomeVoice AI :

對話式家電控制與行為記憶系統



專題指導教授 Independent Study Advisor :

羅壽之

專題參與人員 Team Member :

蔡東廷

林皓奎

謝傑安

楊忠諭

中 華 民 國 115 年 5 月 22 日

國立東華大學資訊工程學系

專題報告原創性聲明

National Dong Hwa University

Department of Computer Science and Information Engineering Statement of Originality

本人鄭重聲明：

所呈交的專題報告是在指導老師指導下進行的研究工作及取得的研究成果。除文中已經註明引用的內容外，本報告不包含任何其他個人或集體已經發表或撰寫過的研究成果。對本文的研究做出重要貢獻的個人與集體，均已在文中以明確方式標明。若有違上述聲明，願依校規處分及承擔法律責任。

I hereby affirm that the submitted project report is the result of research under the supervision of my advisor. Except where due references are made, the report contains no material previously published or written by another person or group. All significant facilitators to the project have been mentioned explicitly. Should any part of the statement were breached, I am subject to the punishment enforced by the University and any legal responsibility incurred.

學號 Student No.	學生姓名 Name	親筆簽名 Signature
411221315	蔡東廷	蔡東廷
411221303	林皓奎	林皓奎
411221235	謝傑安	謝傑安
411221206	楊忠諭	楊忠諭

目錄

專題報告原則性聲明	ii
目錄	iii
圖目錄	v
表目錄	vi
摘要	vii
誌謝	vii
第一章、 前言	1
第二章、 研究動機與研究問題	2
2.1 研究動機與目的	2
2.2 研究問題	3
第三章、 研究方法與步驟	3
3.1 整體架構流程概述	3
3.2 硬體環境與平台	3
3.3 軟體工具與技術	4
3.3.1 Faster-Whisper 語音辨識.....	4
3.3.2 Google Gemini API 與 Tool Calling	4

3.3.3 Piper TTS 語音播放技術	4
3.3.4 CustomTkinter 圖形使用者介面	4
3.4 混合意圖解析機制 (Intent Router)	5
3.5 使用者習慣學習與排程系統	5
3.6 系統實作細節	6
3.6.1 音訊層	6
3.6.2 核心層	6
3.6.3 裝置層	8
3.6.4 圖形介面控制	9
第四章、實驗結果與討論	10
4.1 實驗設計	10
4.2 結果呈現	11
4.2.1 不同 Faster-Whisper 模型效能比較	11
4.2.2 端對端系統延遲分析	13
4.2.3 指令理解動作執行準確率	14
4.2.4 記憶、排程與 GUI 互動測試	15

4.3 討論與分析	16
4.3.1 語音辨識模型的選用取捨	16
4.3.2 本地端 AI 推論為系統延遲的瓶頸	17
4.3.3 雙層路由架構的設計效益	17
五、 研究檢討與未來展望	17
5.1 研究限制與檢討	17
5.2 未來展望	17
5.3 結論	18
參考文獻	19

圖目錄

圖 1、專題架構流程圖	3
圖 2、使用者程式介面	5
圖 3、混合意圖流程圖	5
圖 4、使用者習慣與排程系統流程圖	6
圖 5、使用者規則格式	8
圖 6、各模型 CER 比較	11
圖 7、各模型句級正確率時間比較	12
圖 8、各模型推論時間比較	12
圖 9、PC 與 Pi 各段平均耗時堆疊圖	13
圖 10、FastPath 與 Gemini 路由端對端延遲比較	13
圖 11、路由正確率、動作匹配率、執行成功率比較	14
圖 12、記憶與排程各功能測試通過率	15
圖 13、GUI 各類別指令延遲分佈	16

表目錄

表 1、記憶系統三層儲存架構	7
表 2、硬體配置與 GPIO 腳位對應	8
表 3、四階段實驗總覽	10
表 4、NLU 測試結果彙總	14
表 5、NLU 路由錯誤與動作不匹配主要模式	14

摘要：

本專題以 Raspberry Pi 4 為核心，開發一套中英雙語對話式智慧家居控制系統。系統整合 Faster-Whisper 本地語音辨識、Google Gemini API 語意理解，以及 Piper TTS 語音合成，透過混合意圖解析機制（FastPath 規則式快速路徑 + Gemini 深度理解），實現 LED 與小風扇等模擬家電的即時控制、個人化規則學習與智慧排程功能。在資源受限的嵌入式環境下，我們成功兼顧了反應速度與理解能力。

關鍵字：智慧家居、語音辨識、混合意圖解析、邊緣運算、規則學習、行為記憶

誌謝：

這次專題能夠順利完成，首先要深深感謝教授羅壽之。從最初選題目階段，教授提供了許多不同的研究方向供我們參考，讓我們能找到真正感興趣且具有挑戰性的主題。在整個實作過程中，教授也慷慨提供各式硬體設備讓我們無需擔心資源不足的問題，可以專注於研究本身。尤其在寒假期間，非常感謝教授在百忙之中抽出時間來每週聽取我們的進度報告，給予精準的建議與指引，讓我們在遭遇困難時能夠找到正確的方向。最後，要謝謝一起走過這段旅程的所有組員。大家在面對困難時互相支持、彼此鼓勵，共同解決一個又一個的難題。這段共同奮鬥的經歷，不僅讓我們在技術上有所成長，更讓我們學習到了團隊合作的重要性。

第一章 前言

隨著人工智慧與物聯網技術的快速發展，智慧家居已成為現代生活中重要的應用趨勢。傳統的手動開關或手機 App 控制方式雖然可靠，但仍需使用者手動操作，無法達到自然、直覺的互動體驗。語音控制作為最符合人類溝通習慣的介面，逐漸成為智慧家居的關鍵入口。

然而，目前市面上的商業語音助理多依賴雲端服務，在隱私保護、網路穩定性以及個人化客製化方面仍有明顯不足。特別在台灣家庭環境中，常面臨口音辨識不準、延遲較高以及隱私疑慮等問題。因此，我們希望開發一套以本地運行為主、兼具即時性與智慧判斷能力的語音控制系統。

本專題以 Raspberry Pi 4 為核心硬體，結合本地語音辨識技術與生成式人工智慧，設計一套混合式聲控智慧家居系統。我們使用 LED 模擬燈光、小風扇模擬風扇，並以七段顯示器呈現溫度控制，同時整合個人化規則學習與智慧排程功能，讓使用者可以用自然語言輕鬆設定「幾點開燈」或「我好熱就開風扇」等指令，實現更貼近生活的智慧體驗。

本研究的主要目標是在嵌入式裝置資源有限的情況下，探索如何平衡語音辨識準確率、系統反應速度與功能完整性，並透過混合架構在本地運行為主的基礎上，提供兼顧速度、理解能力與實用性的解決方案。

第二章 研究動機與研究問題

2.1 研究動機與目的

在實際開發過程中，我們發現雖然市面上已有許多語音助理，但大多高度依賴雲端服務，導致隱私外洩風險高、網路不穩時容易失效，且難以針對個人生活習慣進行深度客製化。尤其在台灣家庭環境中，這些問題更加明顯。

因此，我們決定以 Raspberry Pi 4 為平台，開發一套以本地運行為主的對話式智慧家居系統。系統中語音辨識 (Faster-Whisper) 與語音合成 (Piper TTS) 皆在本地執行，而複雜語意理解則透過 Google Gemini API 進行。希望讓使用者能用自然語言控制家電，同時支援個人化規則學習與智慧排程，讓系統真正「記得」每個人的使用習慣。

本研究的主要目的如下：

1. 實作高效的本地語音辨識系統，並支援中英雙語切換。
2. 設計混合式意圖解析機制，在本地快速處理與雲端深度理解之間取得平衡。
3. 實現個人化規則學習與智慧排程功能，提升系統的實用性。
4. 在 Raspberry Pi 4 運算資源有限的情況下，探討模型選擇、路由設計與效能取舍。

2.2 研究問題

我們主要研究的問題有：

1. 結合 Whisper 轉譯文字後，如何快速分析語意並轉換成 GPIO 訊號控制並掌握家電狀態，以達到低延遲的本地語音交互？
2. 挑選並比較不同大小的 Faster-Whisper 模型，分析三個版本模型的系統穩定性以及即時回應的速度，嘗試平衡延遲時間和辨識準確率。
3. 評估本系統在 PC 與 Raspberry Pi 4 硬體環境下，各自不同階段的指令辨識準確率、回應時間與系統穩定性的表現為何？

第三章 研究方法與步驟

3.1 整體架構流程概述

本系統採用分層式架構，使用者透過語音輸入指令，經由音訊層中的 Whisper STT 模型轉換為文字後，傳送至核心層（Router + Agent）進行意圖分類與解析決策。接著，核心層依序透過 FastPath 規則式解析器與 Gemini 語言模型進行雙層路由，並輸出標準化動作指令（JSON 格式）。裝置層透過 Raspberry Pi GPIO 將指令轉換為控制訊號，驅動燈光、風扇等硬體執行，同時讀取 DHT11 感測器的環境數據。最後透過 Piper TTS 合成語音即時回饋執行結果。

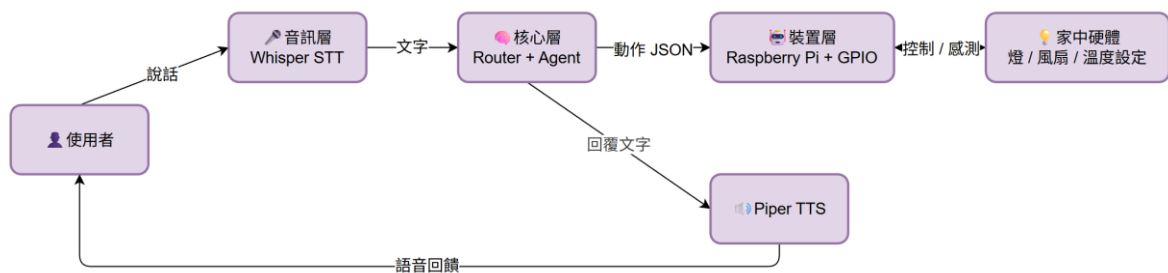


圖 1：專題架構流程圖

3.2 硬體環境平台

本系統以 Raspberry Pi 4 作為主要運算平台。Raspberry Pi 4 具備四核心 ARM Cortex-A72 處理器、最高 8GB RAM，以及豐富的 GPIO 腳位，非常適合開發嵌入式 IoT 入門應用。

硬體部分主要包含：

- Raspberry Pi 4：負責語音處理、意圖解析與控制邏輯。
- USB 麥克風：用於接收使用者語音輸入。
- 揚聲器：用於播放系統回應語音（搭配 Piper TTS）。
- 麵包板與 LED：作為模擬家具的輸出裝置，透過 GPIO 直接控制。
- 電源模組：提供穩定 5V 電源給 Raspberry Pi 與周邊裝置。

由於 Raspberry Pi 4 運算資源有限，我們在模型選擇與系統設計上必須考量效能與功能的取捨，這也是本研究的重要討論重點之一。

3.3 軟體工具與技術

本系統在軟體層面整合了多種開源工具與雲端 API，以實現高效的語音辨識、自然語言理解、語音回應以及圖形介面呈現。為因應 Raspberry Pi 4 運算資源有限的特性，我們在工具選擇上特別注重輕量級與執行效率，同時透過混合架構設計平衡整體效能。

3.3.1 Faster-Whisper 語音辨識

本系統採用 **Faster-Whisper** 作為本地語音辨識引擎。**Faster-Whisper** 是 OpenAI Whisper 模型的優化版本，具有較高的執行效率與較低的記憶體占用，適合在 Raspberry Pi 等嵌入式裝置上運行。本系統選擇 base 模型作為主要方案，並支援中英文混合辨識。

3.3.2 Google Gemini API 與 Tool Calling

本系統整合 **Google Gemini API**，用於複雜語意理解與多輪對話。透過 Prompt Engineering 設計客制化結構指令，要求 Gemini 以 JSON 格式輸出動作指令（包含裝置控制、排程管理、習慣規則學習等），系統再解析此 JSON 並在本地執行對應的控制邏輯，實現更智慧的自然語言處理。

3.3.3 Piper TTS 語音播放技術

系統的語音回應部分採用 Piper TTS 作為文字轉語音引擎。Piper 屬於輕量級且可完全離線運行的 TTS 模型，語音合成速度快，能滿足本研究對即時性的要求。此外，Piper TTS 也能在 Raspberry Pi 上生成自然語音，具有良好的隱私保護與離線可用性。

3.3.4 CustomTkinter 圖形使用者介面

本系統使用 Python 的 **CustomTkinter** 開發現代化的圖形使用者介面（如圖 2）。主要功能包括即時顯示家具狀態、排程清單、使用者習慣記憶清單、提供快捷指令按鈕，以及中英語言與深色淺色介面的切換按鈕。介面設計以簡潔實用為原則，確保在嵌入式環境下操作流暢。

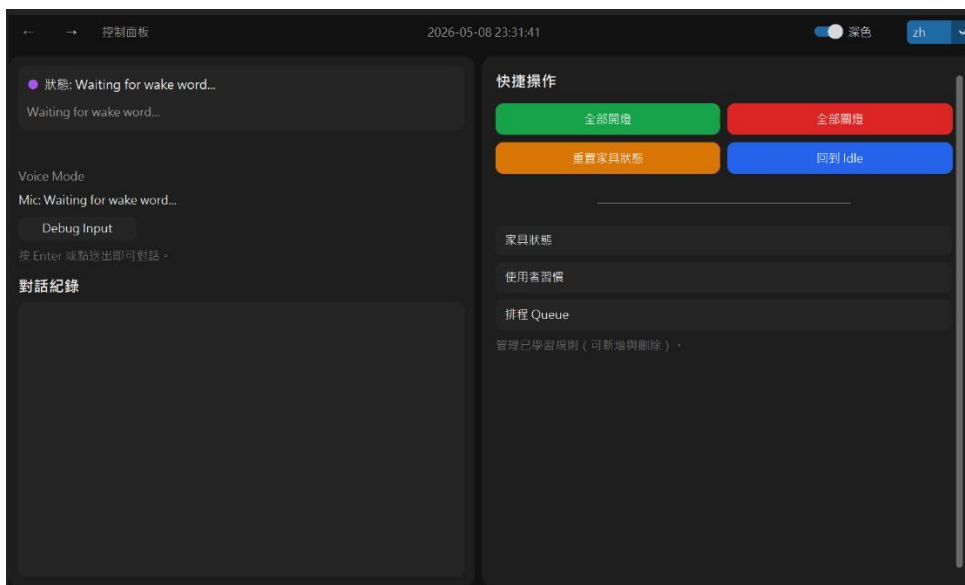


圖 2：使用者介面

3.4 混合意圖解析機制 (Intent Router)

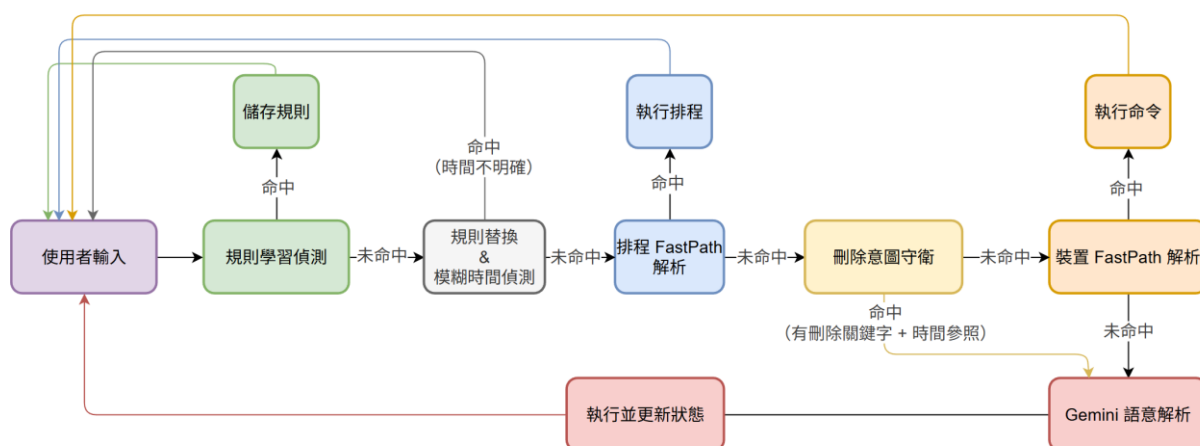


圖 3：混合意圖流程圖

本系統採用混合意圖解析機制，結合「規則式快速路徑 (FastPath) 與 LLM 語意後備路徑」，以兼顧即時性與語意理解能力。如圖 3 所示，系統在處理使用者輸入時，會先進行自訂規則替換，再依序經過以下規則式檢查：

首先進行規則學習句型偵測，判斷輸入是否為「當我說 X 代表 Y」類型的學習指令，若是則直接將對應關係存入規則庫。

接著進入排程解析階段，處理新增、查詢、刪除及啟停等排程相關指令。在此階段，系統會同步進行模糊時間偵測：若偵測到時間詞存在「早上／晚上」不明確的情況，系統會直接在聊天視窗向使用者提問確認，無需經過 Gemini。

其後由裝置 FastPath 處理燈光、風扇、溫度等明確的設備控制指令。此外，系統設有刪除意圖守衛：當輸入中含有刪除關鍵字且伴隨時間參照時，會跳過 FastPath，直接交由 Gemini 判斷是否需要澄清，避免誤刪除。

若以上規則路徑均未命中，系統才會將輸入交由 Gemini 進行語意解析，並輸出結構化的 intent 與 actions。此設計在資源受限的嵌入式環境下，同時兼顧了回應速度、可控性與語意彈性。

以下為實際指令範例，說明系統如何判斷與執行：

- (1) 明確指令（走 FastPath）「開客廳燈」→ 直接被裝置 FastPath 解析器命中，立即執行開燈動作。
- (2) 規則學習句（走規則學習路徑）「當我說我好熱，代表設定溫度 25 度並開啟風扇」→ 系統辨識為學習句型，立即將「我好熱」存入規則庫。
- (3) 排程相關指令（走排程解析器）「每天晚上十點關客廳燈」→ 由排程解析器處理，新增排程任務。
- (4) 模糊時間處理（直接詢問使用者）「幫我設定 8 點開燈」→ 系統偵測到時間詞不明確，直接向使用者確認：「請問是早上 8 點還是晚上 8 點開燈呢？」
- (5) 模糊或複合語意（走 Gemini 路徑）「我現在有點熱，幫我調舒服一點」→ 三道規則均未命中，系統交由 Gemini 處理。此時 Gemini 會先向使用者確認「請問要調到多少度？」，待使用者回應後，再根據實際要求的溫度設定冷氣與風扇。

此外，系統也加入了防誤判設計：當偵測到刪除關鍵字且含時間參照時，會跳過 FastPath，並交由 Gemini 判斷是否需要澄清，避免錯誤執行，以提升穩定性。

3.5 使用者習慣學習與排程系統

為了讓系統能符合每位使用者的個人偏好與生活作息，本專題在核心層中設計了排程管理與自訂規則學習兩個子系統。使用者可透過語音或文字指令新增排程任務（例如：每天晚上六點開燈），系統將在指定時間自動觸發，並且可以在圖形介面中查看與管理現有排程。同時，使用者也能建立個人化的對應觸發詞（例如：「我好熱」→ 設定溫度為 25 度並開啟風扇），讓系統依據使用者的習慣詞彙啟動裝置。

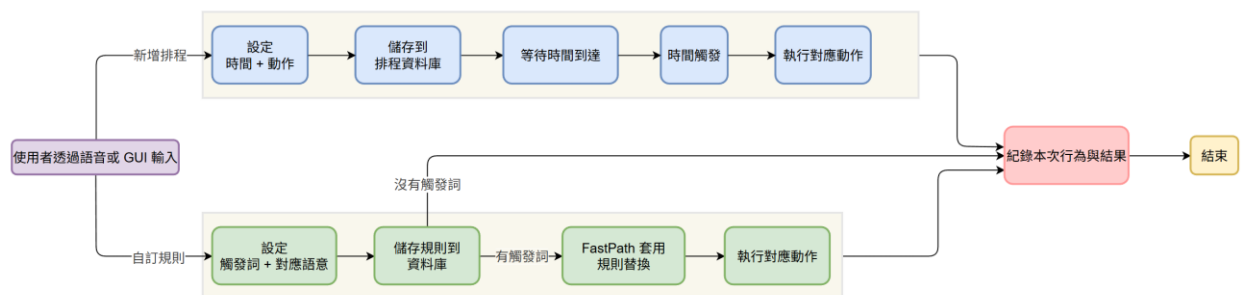


圖 4：使用者習慣與排程系統流程圖

3.6 系統實作細節

3.6.1 音訊層

1. 語音辨識 (STT)：

本系統採用 Faster-Whisper 函式庫搭載 Whisper base 模型執行本地語音辨識，以避免語音資料傳送至外部服務，進而實現本地運行的理念。音訊透過 USB 麥克風進行擷取，當能量超過啟動閾值時開始錄音，降至結束閾值以下並持續 1.2 秒後停止，而錄音長度不足 0.25 秒者，將視為無效輸入並予以捨棄。辨識語言設定為自動偵測，可同時支援中文與英文。

2. 語音合成 (TTS)：

語音回覆採用 Piper TTS 引擎，依系統語言分別載入不同聲音模型：中文使用 voice.onnx，英文則使用 en_US-lessac-medium.onnx。

3.6.2 核心層

1. 混合意圖解析機制：

本系統主要採用混合解析架構，依優先順序逐層處理輸入：

- (1) 文字正規化：修正 Whisper 常見錯誤（如「開登」→「開燈」）。
- (2) 規則比對：以 9 組正規表達式比對學習句型（如「我說 X 代表 Y」），命中則儲存規則並回覆。此步驟置於排程解析之前，避免時間詞造成誤判。
- (3) 排程解析：依序處理新增、查詢、刪除與啟停，全程規則式，不呼叫 LLM。若含刪除關鍵字但無法解析，跳過裝置 FastPath 直接送 Gemini。
- (4) 裝置 FastPath：先套用自訂規則，再過濾疑問句與相對時間輸入，最後合併溫度、風扇、燈光、冷氣四個子解析器，支援「開風扇且設溫度 28 度」等複合指令。
- (5) Gemini 路徑：前述均未命中時使用。透過 Prompt Engineering 提供裝置狀態、最近 5 筆對話紀錄、自訂規則與 11 條系統規則，要求輸出 reply、intent、actions，再依 intent 的類型（command / query / unclear / error）決定後續執行邏輯。

2. 記憶管理機制：

本研究的記憶管理機制，採用的是三層獨立的記憶系統來統一進行管理，如下表 1 所示：

表 1：記憶系統三層儲存架構

儲存層	用途
短期記憶	保留最近 10 筆對話，提供 Gemini 用以判斷上下文
長期記憶	保留所有對話的完整紀錄
規則庫	儲存使用者自訂的觸發詞與對應語意

短期記憶中的每筆紀錄包含時間戳記、使用者輸入與系統回覆。長期記憶僅追加不覆寫，保留完整的對話歷史紀錄。規則庫支援新增、更新與刪除。此外，當進行刪除規則時，系統會同時清除短期記憶中含有該觸發詞的對話紀錄，防止 Gemini 從歷史對話推斷已刪除的規則造成錯誤。

3. 排程管理子系統：

本研究在排程系統中是以 JSON 儲存每筆排程（含時間、重複模式、裝置動作、啟停狀態與最後執行紀錄），最多 5 筆。新增時會進行衝突偵測，當同一裝置在同一時間執行相同動作時，會拒絕新增。此外，背景執行緒每分鐘會負責排程輪詢，符合條件且狀態啟用時會自動觸發。為避免與手動操作衝突，手動執行裝置後，會對該裝置設置 60 秒封鎖期，期間內排程不會重複觸發。

4. 自訂規則學習子系統：

使用者可透過自然語言直接教導系統個人化的觸發詞對應，例如「當我說我好熱，代表設定溫度 25 度並開啟風扇」。

系統識別到這類學習句型後，會將「我好熱」與「設定溫度 25 度並開啟風扇」的對應關係，以鍵值對的形式持久化儲存至規則庫，如下圖 5 所示：



```
1 {
2   "rules": [
3     {
4       "trigger": "我好熱",
5       "meaning": "設定溫度25度並開啟風扇"
6     }
7   ]
8 }
```

圖 5：使用者規則格式

此後，當使用者說出觸發詞「我好熱」時，系統在進入裝置指令解析階段時，會先對照使用者自訂的規則庫進行觸發詞替換，將輸入替換為對應的完整描述，再交由後續的指令解析流程（快速規則比對或 Gemini 語意理解）執行對應動作。

為避免較短的觸發詞誤替換掉較長觸發詞的一部分，替換時會依觸發詞長度由長至短依序比對，確保優先匹配最完整的觸發詞。

3.6.3 裝置控制層

本研究的裝置主要透過 Raspberry Pi 的 GPIO 訊號進行管理。硬體的配置如下表 2 所示：

表 2：硬體配置與 GPIO 腳位對應

模擬家電	設備	GPIO	觸發邏輯
廚房燈	LED 燈（紅）	22	高電位觸發
客廳燈	LED 燈（綠）	10	高電位觸發
客房燈	LED 燈（黃）	9	高電位觸發
電風扇	繼電器與小風扇	16	低電位觸發
冷氣	共陽極七段顯示器	a:2、b:27、c:18、d:15、 e:14、f:3、g:23 digit2:4、digit3:17	低電位顯示
感測器	DHT11	25	背景執行緒每 2 秒讀取一次

3.6.4 圖形使用者介面

GUI 以 Python 的 CustomTkinter 函式庫開發，採左右分割佈局。左側為對話面板，包含指令輸入欄位、對話歷史（最多保留 20 筆，以不同色彩標示使用者 / 助理 / 系統角色）及系統狀態指示燈。右側則為多頁面內容區，提供以下四個頁面：

- 控制面板：快捷操作按鈕（全部開燈、全部關燈、重置狀態、回到待機）及各頁面導覽入口
- 家具狀態：顯示冷氣狀態、溫度設定值、風扇狀態、三區燈光狀態，以及 DHT11 讀取的環境溫濕度
- 使用者習慣：自訂規則清單，支援新增與刪除操作
- 排程清單：現有排程的查看與管理

另外，介面也支援深色與淺色主題切換，以及中英文語言的切換。

第四章 實驗結果與討論

4.1 實驗設計

本研究依系統功能將評估實驗分為四個階段，各階段測試項目、環境與資料量如表 3 所示。

表 3：四階段實驗總覽

階段	測試項目	環境	樣本數	量測指標
階段一	Faster-Whisper 模型效能	PC、Raspberry Pi 4	各模型各 144 筆	CER、句級正確率、推論時間
階段二	端對端系統延遲	PC、Raspberry Pi 4	各 144 筆	各段耗時、端對端延遲
階段三	指令理解與動作執行	Raspberry Pi 4	65 筆	路由正確率、動作匹配率、執行成功率
階段四	記憶、排程與 GUI 互動	Raspberry Pi 4	記憶、排程功能：31 筆 GUI 互動：36 筆	功能通過率、UI 更新延遲、狀態同步率

四個實驗階段依序評估系統的語音辨識基礎效能（階段一）、完整推論的時間開銷（階段二）、自然語言理解與指令執行的準確率（階段三），以及記憶管理、排程調度與圖形介面互動的功能正確性和延遲情況（階段四）。

- 階段一：測試 tiny、base、small 三種 Faster-Whisper 模型的辨識效能與推論時間。
- 階段二：測量從語音輸入到系統回覆的完整端對端延遲，找出系統瓶頸。
- 階段三：使用純文字測試，評估 FastPath 與 Gemini 的指令理解與動作執行正確率。
- 階段四：驗證規則學習、排程管理以及 GUI 操作的穩定性與實際延遲表現。

4.2 結果呈現

4.2.1 不同 Faster-Whisper 模型效能比較

我們在 PC 和 Raspberry Pi 4 兩種平台上，測試了 tiny、base 和 small 三種不同規模的 Faster-Whisper 模型。每種模型各進行 144 筆語音測試，量測指標包含字元錯誤率 (CER)、句級正確率與平均推論時間。CER 的定義為辨識結果與標準答案之間每個字元的插入、刪除、替換錯誤總數除以標準答案字元總數，其公式如下：

$$CER = \frac{S + D + I}{N} \times 100$$

其中 S 為替換字數， D 為刪除字數， I 為插入字數， N 為標準答案之總字總數。而句級正確率則可使用如下的公式來呈現：

$$Sentence Accuracy = \frac{\text{正確預測的句子總數}}{\text{測試樣本的句子總數}} \times 100\%$$

由圖 6 至圖 8 的結果顯示，模型規模越大，辨識準確度越高，但推論時間也明顯增加。在 PC 端，三種模型的推論時間都較短；但在 Raspberry Pi 4 上，small 模型雖然 CER 最低、句級正確率最高，卻需要 16.5 秒才能完成辨識，已不適合即時互動。因此，我們最終選擇 base 模型，它在 Raspberry Pi 4 上將推論時間縮短約 50%，在準確度與實際使用體驗之間取得較好的平衡。

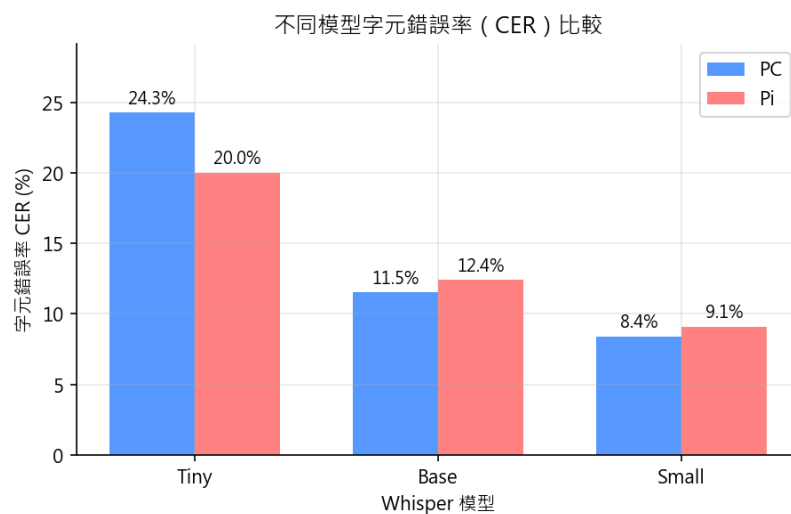


圖 6：各模型 CER 比較

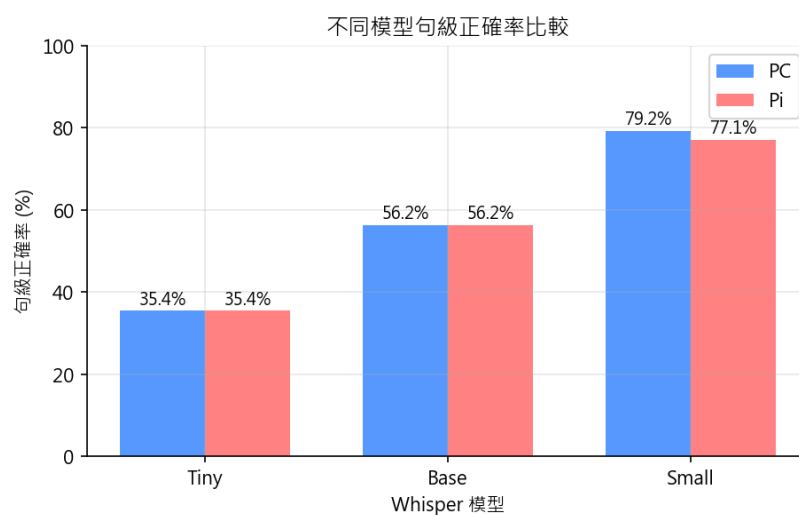


圖 7：各模型句級正確率比較

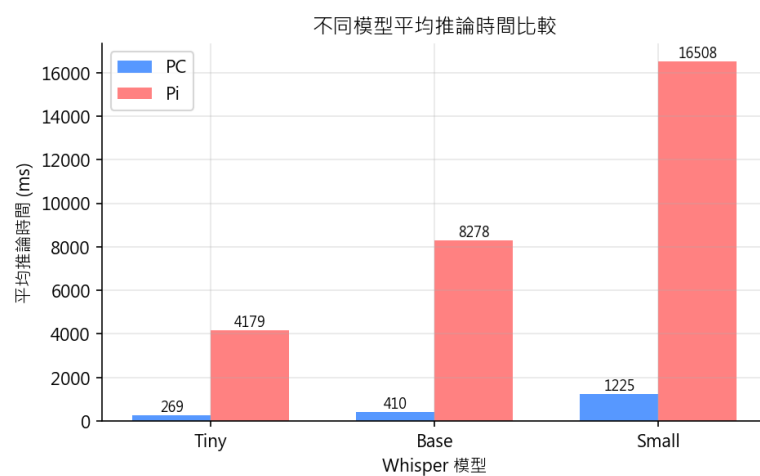


圖 8：各模型推論時間比較

4.2.2 端對端系統延遲分析

我們使用與階段一相同的 144 筆語音資料，完整測量從語音輸入到系統回覆的端對端延遲。

結果顯示（圖 9），STT（語音轉文字）和 TTS（文字轉語音）是系統最主要的延遲來源，在 Raspberry Pi 4 上這兩個階段合計佔總延遲的 94.8%。

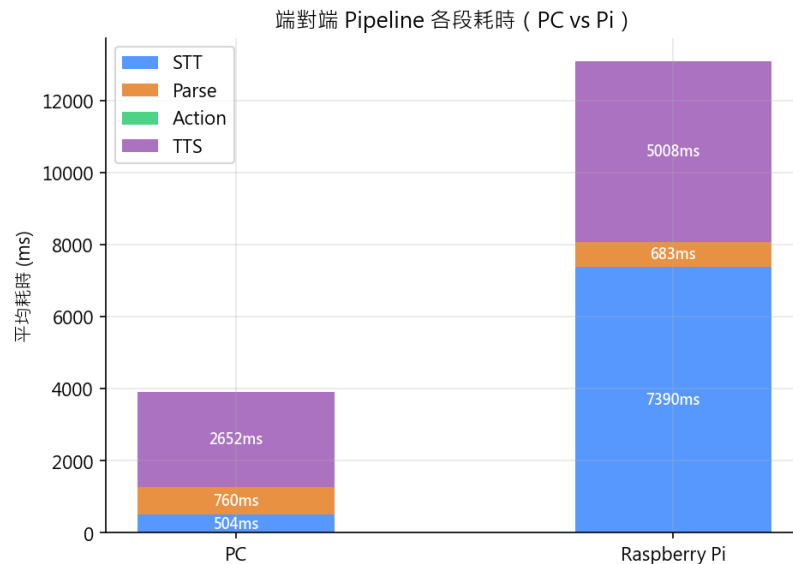


圖 9：PC 與 Pi 各段平均耗時堆疊圖

（圖 10）進一步比較兩種路由，Gemini 路由明顯比 FastPath 慢許多。這表示本地 AI 模型推論才是目前系統速度最大的瓶頸，即使優化解析或網路部分，幫助也有限。

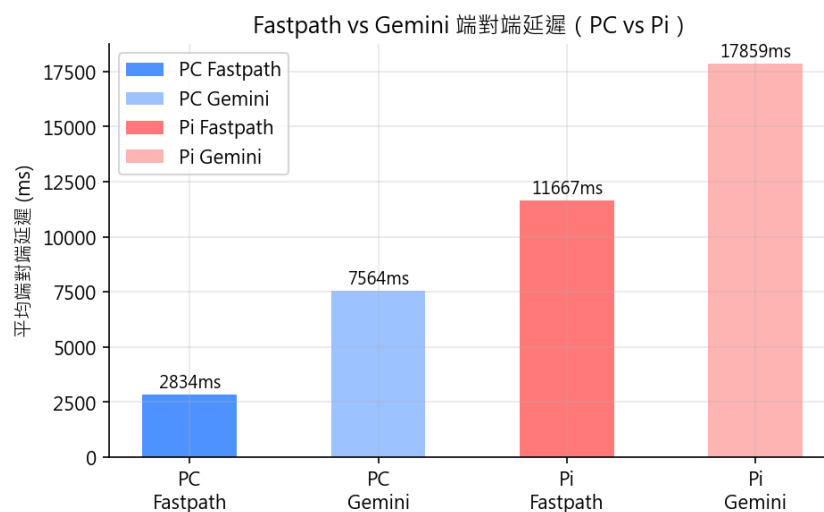


圖 10：FastPath 與 Gemini 路由端對端延遲比較

4.2.3 指令理解與動作執行準確率

我們使用 65 筆純文字測試案例（排除 STT 辨識誤差），評估 FastPath 與 Gemini 兩種路由的自然語言理解能力。測試案例包含明確指令、複合指令、口語化表達以及模糊意圖四種類型。

表 4：NLU 測試結果彙總

範圍	樣本數	路由正確率	動作匹配率	執行成功率
整體	65	92.3%	86.2%	100%
FastPath	43	88.4%	90.7%	100%
Gemini	22	100.0%	77.3%	100%

表 4 顯示，整體執行成功率達到 100%，代表系統能可靠地將指令轉換為裝置動作。但路由正確率只有 92.3%，動作匹配率為 86.2%，仍有改善空間。後續我們分析錯誤案例

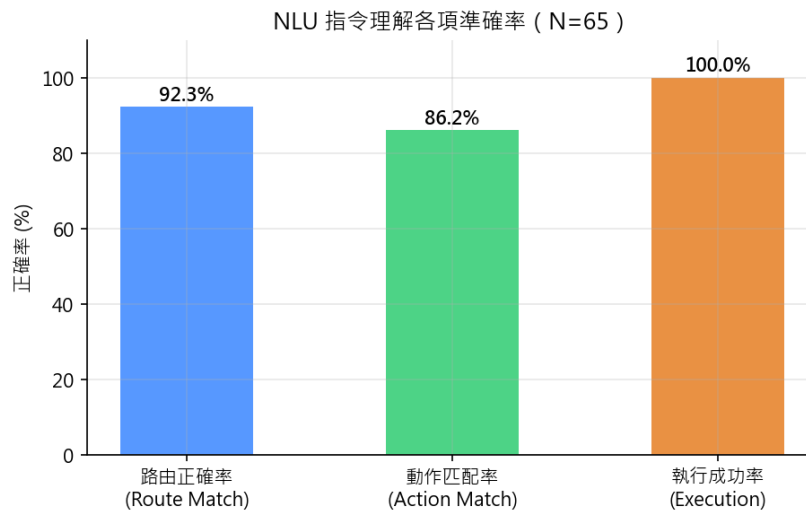


圖 11：路由正確率、動作匹配率、執行成功率比較

因此，針對上述（圖 11）路由正確率與動作匹配率之缺口，本研究進一步針對錯誤案例進行分析，探究系統在語意解析與路徑分流時的瓶頸。下表 5 歸納了測試過程中觀測到的主要錯誤模式、產生原因以及代表之案例。

表 5：NLU 路由錯誤與動作不匹配主要模式

錯誤模式	代表案例（輸入 → 實際行為）	說明
關鍵字衝突	客廳燈開著嗎？→ FastPath 攔截並開燈	詢問語句包含控制關鍵字，導致 FastPath 錯誤攔截
自訂規則判斷失效	當我說睡覺模式就關掉所有設備 → FastPath 誤攔截將所有設備關閉	本地規則無法精確解析設備控制與自訂規則的差異，導致結果錯誤
語意模糊與過度推論	把溫度調高一點 → Gemini 產生產生 JSON 值為當前溫度加 1 度	雖走正確路由，但 Gemini 會主動推論，而非詢問使用者

最後，針對實驗中所觀察到的三種主要錯誤模式，本研究最後對程式碼做了以下幾點改進：

1. **關鍵字衝突**：針對關鍵字衝突的問題，本研究主要是在 FastPath 比對關鍵字前，會先以正規表達式比對疑問字或符號（如：嗎、？）以及疑問詞清單進行意圖預判，並在偵測為疑問句時，交給 Gemini 來進行處理，以避免疑問句被誤判為執行指令。
2. **自訂規則判斷失效**：針對自訂規則判斷失效的問題，本研究則是使規則學習句的處理函式的優先順序高於所有裝置指令解析處理的函式。一旦判定為規則學習句則立即儲存規則並返回，避免 FastPath 的裝置關鍵字被誤判而觸發。
3. **語意模糊與過度推論**：針對語意模糊與過度推論的問題，本研究則是在傳送給 Gemini 的指令中，明確規定「指令若意圖模糊，請向使用者提出澄清問題，而非自行推斷數值」，確保不產生使用者未授權的動作。

4.2.4 記憶、排程與 GUI 互動測試

我們使用 30 筆記憶與排程功能測試，以及 36 筆 GUI 互動測試，驗證規則學習、排程管理和圖形介面的實際表現（測試前皆進行狀態重置，使用最終修正後的程式碼）。

圖 12 顯示，排程的新增、查詢、刪除等功能與規則學習成功率均達到 100%，僅習慣觸發成功率為 88%。

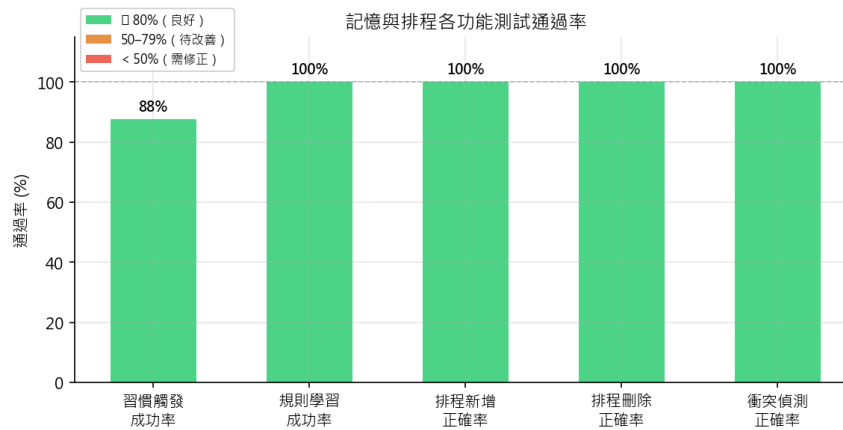


圖 12：記憶與排程各功能測試通過率

GUI 測試以自動腳本依序執行 36 道指令，模擬使用者透過圖形介面控制智慧家居設備的完整操作流程，前 22 道為明確的控制指令（預期走 FastPath），後 14 道則為口語化或複合語意指令（預期走 Gemini）。

圖 13 則顯示，直接透過 GUI 操作的 FastPath 路由延遲很低（平均 845 ms），而 Gemini 路由的延遲明顯較高（平均 2325 ms）。這再次確認雙層路由設計的必要性。

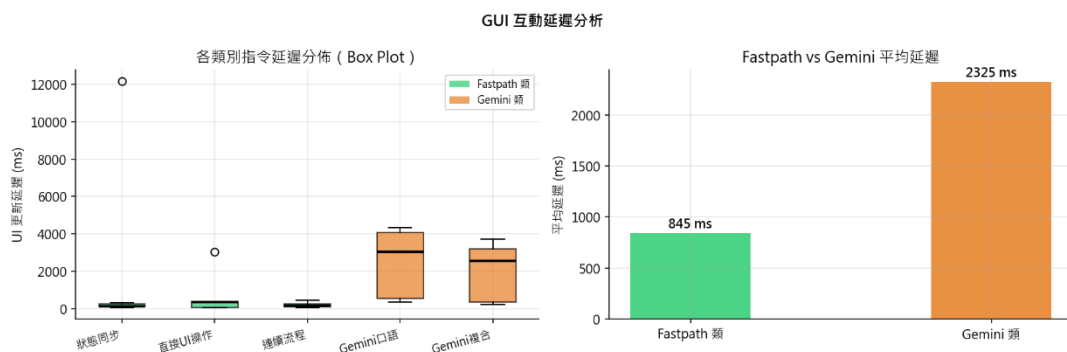


圖 13：GUI 各類別指令延遲分佈

4.3 討論與分析

本研究透過四個實驗階段的數據採集，針對系統效能、延遲成因及路由策略進行深層分析，探討結果如下：

4.3.1 語音辨識模型的選用取捨

實驗結果顯示，模型規模越大，辨識準確度越高，但推論時間也越長。在 Raspberry Pi 4 上，small 模型雖然最準，卻因為延遲太高而無法實際使用。我們最後選擇 base 模型，主要是考量在硬體資源有限的情況下，必須同時兼顧準確度和即時性。

4.3.2 本地端 AI 推論為系統延遲的瓶頸

端對端延遲分析顯示，STT 和 TTS 兩個階段佔了總延遲的絕大部分，尤其在 Raspberry Pi 4 上更明顯。這表示即使我們改善指令解析或網路部分，對整體使用體驗的幫助其實有限。因此，我們認為未來優化的重點應該放在本地 AI 推論的加速上。

4.3.3 雙層路由架構的設計效益

從 NLU 測試和 GUI 延遲結果來看，FastPath + Gemini 的雙層路由設計是合理的。FastPath 可以快速處理明確指令，Gemini 則負責比較複雜或模糊的語意。雖然一開始在路由正確率和動作匹配率還有一些落差，但我們後續針對錯誤案例調整程式後，整體表現已有明顯改善。這個混合架構讓我們在 Raspberry Pi 4 上，同時兼顧了速度與理解能力。

第五章 研究檢討與未來展望

5.1 研究限制與檢討

本研究在實作與測試過程中，確實受到硬體與技術條件的限制。主要問題有兩點：

首先，我們使用的 Raspberry Pi 4 運算資源有限，無法負荷更大規模的 Whisper 模型（如 medium 或 large），導致在追求更高準確度時會遇到 CPU 過載與延遲增加的問題。其次，受限於麥克風硬體規格，在較吵雜的環境下，系統偶爾會出現漏聽或辨識不準的情況。這也凸顯出嵌入式裝置開發時，效能、成本與穩定性之間需要進行取捨。

5.2 未來展望

雖然本專題已成功實作出一個具備語音控制、排程管理與個人化習慣學習的智慧家居系統，但在功能擴展性、穩定性與實用性上仍有改善空間。未來主要發展方向如下：

1. **提升離線能力**：目前複雜語意仍需依賴 Google Gemini API，未來計畫導入量化後的本地大型語言模型（如 Llama 或 Gemma），讓系統能在 Raspberry Pi 上完全離線運行，進一步提升隱私保護與網路穩定性。
2. **擴展至真實家電控制**：目前僅使用 GPIO 控制模擬 LED，未來希望透過 MQTT 通訊協定結合 Node-RED，連接真實的燈具、風扇與空調，讓系統更接近實際居家應用。
3. **強化個人化與安全性**：強化個人化與安全性：目前系統將所有使用者視為同一身分，未來可加入語者識別功能，讓不同家庭成員擁有各自的習慣設定，並拒絕陌生人喚醒。

透過上述改進，本系統預期能從目前的原型階段，逐步發展成為真正離線、個人化且可實際部署於家庭環境的智慧語音助理。

5.3 結論

本專題以打造可實際用於家庭的智慧語音助理為目標，在 Raspberry Pi 4 上成功整合本地語音辨識、混合意圖解析、個人化規則學習與智慧排程等功能。透過 FastPath 與 Gemini 的雙層路由設計，我們在資源受限的嵌入式裝置上，實現了速度與理解能力的平衡。

雖然硬體條件有限，但本研究不僅完成了可實際運作的原型系統，也為後續在邊緣裝置開發類似應用提供了實務經驗與參考方向。

參考文獻

張志豪 (2025)。結合語音輸入與大型語言模型指令翻譯之樹莓派智慧家庭控制系統 (碩士論文)。 <https://hdl.handle.net/11296/y9y8y9>

陳昱愷 (2020)。以 Raspberry Pi 4 來設計家庭環境監控系統之研究 (碩士論文)。 <https://hdl.handle.net/11296/m63enq>

李柏希 (2025)。智慧型出席狀態監控管理系統 (碩士論文)。 <https://hdl.handle.net/11296/mxs756>

金翼 (2016)。研究以樹莓派為平台之智慧監控系統實作 (碩士論文)。 <https://hdl.handle.net/11296/s4v59t>

李建霖 (2014)。以嵌入式 Linux 建構低成本分散式智能家居系統 (碩士論文)。 <https://hdl.handle.net/11296/vn8bfw>

呂智揚 (2018)。互動式語音介面之智慧家庭系統 (碩士論文)。 <https://hdl.handle.net/11296/2ncd6m>

蔣仲緯 (2022)。以物聯網為基礎之智慧家電建置研究 (碩士論文)。 <https://hdl.handle.net/11296/eebzg2>

Ats (2024)。Real-time Speech-to-text on Raspberry Pi and Python。取自 <https://reurl.cc/LQL1Wy>

ggml-org (2024)。whisper.cpp [v1.8.2]。GitHub。取自 <https://github.com/ggml-org/whisper.cpp>

ANAVI Technology (2024)。Video Tutorial: Offline Speech-to-Text on Raspberry Pi 5 with ANAVI Dev Mic。取自 <https://reurl.cc/1k7QkG>

IOT DESIGNPRO (2020)。Speech Recognition on Raspberry Pi for Voice Controlled Home Automation。取自 <https://reurl.cc/9bLz0X>